Aarhus University

Project number - 2026F25

Level Logic

Students:
Thea Teresa Lindgaard (202204350)
Niels Jakob (202105879)
Kasper Stecher (202100451)
Kasper Bentsen (202103249)

Counsellor:
Lukas Esterle

Aarhus, May 29th 2026

Contents

Author: Thea Teresa Lindgaard

1. Introduction

Level logic will be used to describe systems or game elements that are relevant to the level flow. This means things such as level resets, level loading and initialization, level completion, tracking level data and similar. In this document, a description of level logic systems including design, implementation and testing can be found.

1.1. Requirements

The systems described in this document are partially or fully responsible for fulfilling the following requirements:

- E2-US4: Asymmetric Perspectives
- E3-US1: Level Timer
- E3-US2: Time Trial
- E3-US3: Level Failure
- E3-US4: Level Failure Restart
- E3-US5: Player Life System
- E3-US6: Damage From Hazards
- E6-US2: Auto Save

2. Level Manager

The level manager is responsible for level initialization, and acts as a controller for level flow, including resets, completions and failure.

2.1. Design

The goal for the level manager was to centralize game flow logic and track time spent in the level. Each level should have exactly one level manager, that performs level initialization and handles events that affect level flow. The design constraints for the level manager are as follows:

1. Each level is limited to one level manager
2. The level manager handles level initialization
    - Positioning players at spawn locations
    - Triggering state transitions for asymmetric perspectives
    - Starting a level timer
3. The level manager handles level flow events
    - Completions
    - Resets/failure

2.2. Implementation

The following sections describe how the level manager was implemented.

2.2.1. Level Initialization Steps

Most of the level manager's responsibilities are related to level initialization. The initialization steps can be seen in **Figure 1**.

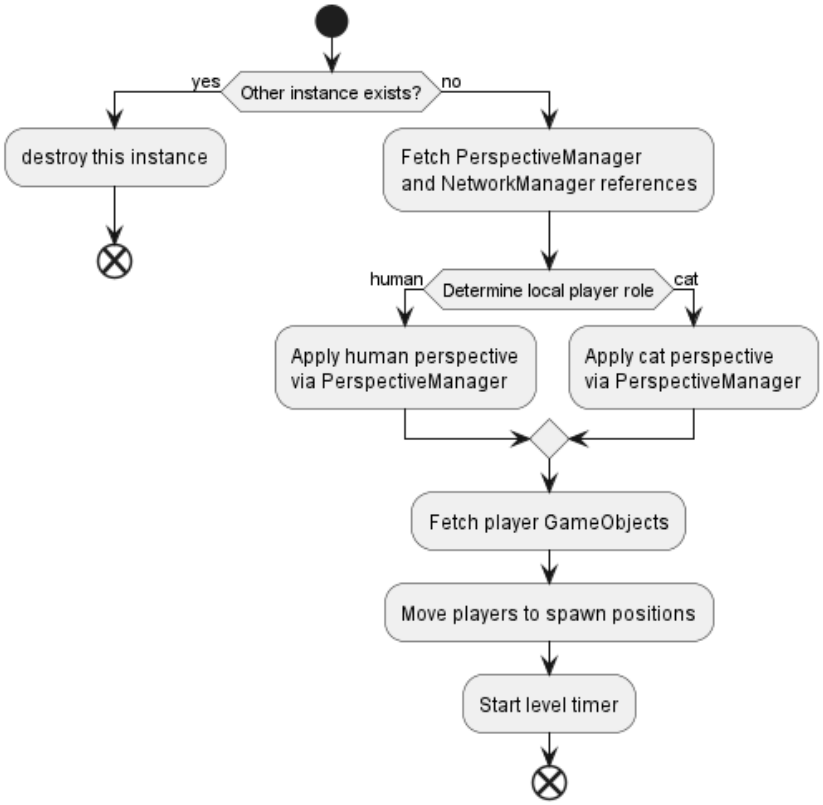

Figure 1: A simplified flow diagram showing the initialization steps performed by the level manager. Error handling steps have been excluded for clarity

The level initialization takes place in the Awake and Start life-cycle event. This ensures that level initialization happens automatically, and with a consistent timing.

Awake is used to resolve references, and ensure that only one level manager exists. If any reference is missing, an error is logged, and the method returns, as errors in this stage of initialization indicates a configuration error in the level.

In Start, the steps performed by the level manager are:

- The level manager uses the network manager from Netcode for GameObjects to find the local player object, and uses that object to determine which role the local player is playing. This is required for the asymmetric perspective to initialize correctly.

- Both player game objects are located and positioned at spawn locations. Spawn locations are configured using empty game objects in the scene for easier level construction
- A stopwatch is started, that will be used to track how long it takes players to complete the level

2.2.2. Singleton

The level manager is implemented using a slightly modified version of the singleton design pattern. Unlike traditional singletons whose constructor is private to prevent multiple instances from being created, the level manager has special handling in the Unity life-cycle event Awake, to ensure it is the only manager in the scene. This is because the Unity Engine does not use or support C# construction to create Components, this is done internally in the Unity Engine**[1]**. In addition, multiple level manager instances can be added to any scene through the Unity Editor, an thus runtime handling is required to enforce a single instance.

The modified singleton pattern uses Awake to detect if any other instances of the level manager exists. If that's the case, the level manager destroys itself. This will ensure that only one level manager will remain after level initialization.

```
private void Awake()
{
    if (_instance != null && _instance != this) Destroy(gameObject);
    else _instance = this;

    // ... other initialization steps ...
}
```

3. Game Zones

Game zones are areas within the game that trigger an event when players enter it.

3.1. Design

The goal for the game zones was to create a modular and flexible way to introduce various types of zones, that trigger some concrete event.

The design constraints for the zones are:

1. Zones can have constraints
   - Optionally require both players in the zone
   - Optionally have a countdown duration before event triggers
2. Extensibility, adding new zones should have no effect on existing zones

3.2. Implementation

The following sections describe how the zones are implemented.

3.2.1. Base Zone

As zone logic is largely the same for most types of zones, a base class was created to handle common logic. All zones utilize the Unity events OnTriggerEnter and OnTriggerExit to detect when a player object interacts with the zone.

The base zone implements common logic to check if the object colliding with the zone is a valid player. Players are marked with the “Player” tag. Additionally, the player must be the local player.

This is because both clients will detect the collision between the player object and the zone. To prevent both game clients from triggering the event, resulting in duplicate events, only the client whose player entered the zone is allowed to trigger the event.

```
private bool IsValidPlayer(GameObject obj)
{
   if (!obj.CompareTag(GameConstants.PLAYER_TAG)) return false;
   if (obj.TryGetComponent(out NetworkObject netObj))
   {
       return netObj.IsOwner;
   }
   return false;
 }
```

The base zone provides override-able methods OnPlayerEnter and OnPlayerExit, that concrete zones can implement. This design separates player detection logic from event logic, promotes code reuse, and improves extensibility.

3.2.2. GroupActivationZone

As multiple zones types have player constraints, another base class GroupActivationZone was implemented based on the above described base class. This base class implements common logic for tracking how many players are in the zone at any time, a countdown period for activating the event, and has handling to ensure both players receive the event.

The countdown was added specifically to group zones, because some zones are intended as a decision making framework for players. The countdown was added to give players time to back out, or to act as a grace period for players who unintentionally entered a zone.

The method below is using RPCs to ensure an authoritative server state. When a player enters a group activation zone, the method below is invoked. The method adds the player to a collection of players inside the zone, and checks conditions for starting the countdown. An equivalent method exists for when a player exits a zone, potentially causing the countdown to be cancelled.

```
[Rpc(SendTo.Server, InvokePermission = RpcInvokePermission.Everyone)]
private void EnterZoneServerRpc(RpcParams rpcParams = default)
{
   ulong clientId = rpcParams.Receive.SenderClientId;
   _players.Add(clientId);

   // check if countdown may start
   if (!IsServer) return;
   if (_players.Count != GameConstants.MAX_PLAYERS) return;
   if (_countdownCoroutine != null) return;

   // start the countdown
   Debug.Log("Countdown start");
   _countdownCoroutine = StartCoroutine(StartCountdown());
   StartCountdownClientRpc(countdownSeconds);
   if (voteText != null)
       UpdateUIClientRpc(_players.Count);
}
```

This ensures that the event is triggered by the server only, and the server shares its state with clients. This way, all players get the event at the same time, and the risk of de-syncs and divergent zone states is eliminated.

The server maintains the authoritative countdown, while broadcasting countdown start to both clients and host.

```
[Rpc(SendTo.ClientsAndHost)]
private void StartCountdownClientRpc(int seconds)
{
    StartCoroutine(ClientCountdown(seconds));
}
```

This countdown is used to display the countdown graphically in the game.

3.2.3. Concrete Zones

Players might encounter the following zones in the game:

- Vote zone: requires both players, and has a countdown period. This zone is what players use in the lobby to enter levels.
- End zone: requires both players, no countdown. This zone is where players must go to complete a level.
- Kill zone: only one player needed, instant. When a player enters a kill zone, they fail the level, and must start over.

Figure 2 illustrates which base classes each zone type is based on.

Figure 2: A class diagram showing all concrete zone implementations

The vote zone utilizes the SceneLoader to transition from the lobby to the level being voted for.

The kill zone and end zone both use the level manager to trigger resets and level completions respectively.

New zone types can be added by implementing these base classes, as all detection and server state logic is handled in the base classes. Overall, this design promotes code reuse, and follows the single responsibility principle, as player detection, network management, and concrete zone event logic is implemented separately.

3.2.4. Trade-offs

While this architectural design is well suited for the game's needs, it does come with drawbacks. Both base zones are tightly coupled to the networking library, as they both perform checks using the network manager. This means that it will be more difficult to replace the networking library later in development, as these classes will have adjust to the change. Thanks to the base classes handling all the network-dependent logic, the consequences of replacing the networking library is limited to these base classes.

Additionally, the concrete zones are also dependant on external classes, and reference other managers as part of the event logic, again resulting in tight coupling.

The vote zone is dependent on the SceneLoader, which was built specifically as a tool to load Unity scenes in a multiplayer context. As the vote zone's sole purpose is to load level scenes, this dependency is natural and acceptable. Additionally, the SceneLoader functions as a sort of interface between the networking library and the zone, further decoupling the zone itself.

The end zone and kill zone are dependent on the level manager to trigger level restarts and completions. Similarly to the vote zone, these dependencies are considered natural and acceptable.

4. Test

The following sections describe how the system was tested, and the results.

4.1. Testing Method

The game systems described in the previous sections are highly dependent on networking and Unity integrated features. This makes typical testing frameworks unsuitable for verifying these systems.

Instead, these game systems were tested using manual playtesting and integration testing. Due to the nature of these systems, they are tightly coupled, and thus hard to fully isolate for testing purposes. Playtesting manually in-game is appropriate and sufficient, as the scope of responsibilities these systems cover is relatively limited.

The systems are tested on a per zone basis, because the zones functions as the entrypoints for all level flow logic described.

The following sections describe the action taken to test the systems, as well as expected results, and whether or not the tests passed. Notice that the testing performed at this stage is not an exhaustive search for rare bugs, but a simple test to confirm basic functionality. A more exhaustive test will be performed later in play-tests done by external players.

4.2. Vote Zone

The vote zone covers the following logic to be tested:

1. Vote zone player requirement
2. Vote zone countdown
3. Vote zone event, loading the corresponding level
4. Level initialization upon entering a level
    - positioning players at spawn locations
    - activation of asymmetrical perspective
    - starting a level timer

Action	Expected	Result
One player enter vote zone	Level does not load	pass
Second player enters vote zone and leaves before countdown reaches zero	Countdown starts, then stops. Level does not load	pass
Second player enter vote zone again and stays	Countdown starts over, level loads after countdown. After level has loaded, players have spawned in different positions, and their perspectives do not match	pass

Notice that the timer is tested in the following section, as the time will be displayed upon level completion.

4.3. End zone

The end zone covers the following logic to be tested:

1. End zone player requirement
2. End zone triggers instantly
3. End zone sends players back to lobby
4. Level manager saves timer data

Action	Expected	Result
One player enter end zone	Level does not end, players remain in level scene	pass
Second player enter end zone	Players are returned to the lobby immediately, the level now has a completion time, as well as appropriate trophies	pass

4.4. Kill zone

1. Kill zone triggers instantly
2. Kill zone has no player requirement
3. Level is reloaded, causing level initialization to occur again

Action	Expected	Result
One player enter kill zone	Level is reloaded, and returned to the state it was in right after entering	pass

5. Conclusion

This document presented the design, implementation and testing of the level logic systems. The systems were designed with a focus on modularity and extensibility. For zones, this was achieved by separating common zone logic form zone event logic, making it possible to extend the game with new types of zones without requiring any changes to external systems.

The level manager handles all common level flow, thus centralizing level flow control. With the modified singleton pattern, duplicate level managers are avoided, which in turn eliminates the risk of duplicate level flow events.

Utilizing Unity's life cycle events made most flow easy to set up, as it is automatically executed at the correct points in time during the game flow.

The system does have tight coupling given the nature of the zones. This is an acceptable trade-off given the zones responsibility, and as all concrete zone classes are very small, it is a manageable risk.

Bibliography

[1] h. Unity huh, “MonoBehvior Construction.” [Online]. Available: https://unity.huh.how/runtime-errors/monobehaviournew.html